

Basic Abstractions in Distributed Algorithms IV

Based on: Cachin, Guerraoui, Rodrigues

Danilo Kačanski

Faculty of Technical Sciences, Novi Sad
Applied algorithms in Control Systems

April 2026.

Examples

Example 1: Ping-Pong with Perfect Links

- This example demonstrates:
 - basic message passing
 - request-response interaction
 - nondeterministic scheduling
- Two processes:
 - PingProcess
 - PongProcess
- Communication uses:
 - Fair-loss link
 - Stubborn link
 - Perfect link (composition)

Goal

Show how reliable communication emerges from layered abstractions.

PingProcess

- On INIT:
 - sends first PING(0)
- On PONG:
 - increments counter
 - sends next PING
 - stops after reaching limit

PongProcess

- On PING(x):
 - replies with PONG(x)

Key Idea

This is a simple **request-response protocol**.

Runtime Configuration

Link composition

```
fll := link.NewFairLossLink(0.0)
sl  := link.NewStubbornLink(fll, 5)
pl  := link.NewPerfectLink(sl)
```

Runtime setup

```
fm := failures.NewNoFailure()
rt := runtime.NewRuntime(42, pl, fm, 30)
```

- no failures
- deterministic execution (seed = 42)
- perfect link ensures:
 - no loss
 - no duplication
 - eventual delivery

Checkers Used

Adding checkers

```
rt.AddChecker(NoDuplicationChecker())  
rt.AddChecker(ReliableDeliveryChecker())
```

- **No Duplication**

- ensures each message is delivered at most once

- **Reliable Delivery**

- ensures all sent messages are eventually delivered

Why these two?

They correspond exactly to the guarantees of a **Perfect Link**.

Execution Flow

- 1 System injects INIT to Ping
- 2 Ping sends PING(0)
- 3 Pong receives and replies PONG(0)
- 4 Ping receives and continues
- 5 Process repeats until limit is reached

What the runtime shows

- message scheduling order
- delivery events
- interaction between processes

Key Idea

Even in a simple protocol, execution is a sequence of events controlled by the runtime.

Example 2: Counter with Fair-Loss Link

- This example demonstrates:
 - message loss
 - difference between safety and liveness
 - why fair-loss links are not enough for reliable systems
- Two worker processes each send **5 INCREMENT** messages
- A counter process increments once for every received message

Expected vs Actual

- Expected: counter = 10
- Actual: counter may be **less than 10** because messages can be lost

Key Idea

This example shows that **correct processes** are not enough if the communication link itself is unreliable.

Processes in the Counter Example

CounterProcess

- Maintains one integer state: Count
- On each INCREMENT message:
 - increases the counter by 1

WorkerProcess

- On INIT:
 - sends a fixed number of INCREMENT messages
- In this example:
 - worker-A sends 5
 - worker-B sends 5

Key Idea

Workers generate the workload, while the counter only reacts to delivered messages.

Link and runtime

```
fll := link.NewFairLossLink(0.5)
fm  := failures.NewNoFailure()
rt  := runtime.NewRuntime(42, fll, fm, 50)
```

- FairLossLink(0.5) means:
 - every message has a 50% chance of being lost
- No process failures are used here
- So the only problem comes from the **communication channel**

Important

The processes are correct, but the network is unreliable.

Execution Flow

- 1 System injects INIT into worker-A and worker-B
- 2 Each worker sends 5 INCREMENT messages
- 3 Every outgoing message passes through the fair-loss link
- 4 Some messages are:
 - enqueued and delivered
 - dropped before delivery
- 5 Counter increments only for messages that actually arrive

Consequence

The final counter value depends on how many messages survive the lossy link.

Safety vs Liveness in This Example

Safety still holds

- the counter will almost never exceed 10
- no extra messages are invented
- message loss does not break **no creation**

Liveness may fail

- some messages may never reach the counter
- therefore not all sent messages are eventually delivered

Final Thought

Fair-loss links can preserve some safety properties, while still breaking liveness.

Example 3: Authenticated Links and Tampering

- This example demonstrates:
 - message authentication using MACs
 - detection of tampering in transit
 - security guarantees of authenticated links
- Two phases:
 - honest communication
 - network attacker modifies messages

Goal

Show that authenticated links ensure messages are not modified without detection.

Processes in the Example

SenderProcess (Alice)

- On INIT:
 - sends one message:

SECRET: "transfer \$100 to Bob"

ReceiverProcess (Bob)

- On SECRET:
 - increments received count
 - stores message content

Key Idea

Bob trusts only messages that pass authentication.

Communication Stack

Link composition

```
f1l := link.NewFairLossLink(0.0)
s1  := link.NewStubbornLink(f1l, 5)
p1  := link.NewPerfectLink(s1)
a1  := link.NewAuthenticatedLink(p1, keys)
```

- Fair-loss → models unreliable network
- Stubborn → retransmits messages
- Perfect → removes duplicates
- Authenticated → verifies message integrity

Key Idea

Authentication is an additional layer on top of reliability.

Phase 1: Honest Communication

- Alice sends a message to Bob
- Message is authenticated using a MAC
- No attacker modifies the message

Result

- message is delivered successfully
- MAC verification succeeds
- Bob accepts and processes the message

Key Idea

Authenticated links behave like perfect links when no attack is present.

Phase 2: Network Attacker

Interceptor

```
rt.SetInterceptor(func(msg process.Message) process.Message {  
    if msg.Type == "SECRET" {  
        tampered := msg.Clone()  
        tampered.Data = "transfer_$10000_to_Eve"  
        return tampered  
    }  
    return msg  
})
```

- Attacker modifies message content in transit
- Original MAC was computed on original data
- Modified message no longer matches MAC

Key Idea

The attacker cannot recompute a valid MAC.

What Happens During the Attack

- 1 Alice sends authenticated message
- 2 Attacker modifies message content
- 3 Message reaches authenticated link
- 4 MAC verification fails
- 5 Message is **rejected**

Result

Bob receives **no message**

Key Idea

Tampering is detected before delivery to the process.

Security Property: Authenticity (AL1)

Authenticity Property

- If a message is delivered:
 - it was sent by the claimed sender
 - it was not modified in transit

What we observe

- Honest case → message delivered
- Tampered case → message rejected

Key Idea

Authenticated links protect integrity and origin of messages.